Latent Representation of Inputs: A Defense Against Adversarial Examples in DQN

Bibek Poudel
bpoudel@memphis.edu
Department of Computer Science
The University of Memphis, Memphis, TN, USA

***Abstract*— Similar to classifiers, deep reinforcement learning (RL) policies are known to be vulnerable to adversarial examples. This might intrigue concerns about widespread adoption of RL in safety critical applications. Under such applications, it may be within reason to expect the agent to choose correct even in the presence of an adversary. To that end, we a adopt existing techniques from image classification namely, latent representation of inputs, more specifically, feature squeezing, to nullify the effects of the modifications in observations caused by a white-box adversary. We modify the input processing pipeline such that the agent is successfully able to choose correct actions even when the observations are perturbed. (See Appendix for videos).**

I. Introduction

Reinforcement Learning (RL) in recent years has achieved super human performance in control and manipulation tasks that often require long term planning, such as autonomous driving and Atari games. These advancements rely on RL policies that are capable of learning directly from observations to produce a distribution over actions to take. Often, such policies are parameterized by neural networks [1].

The threat of adversarial examples [2] to neural networks has been widely studied in the case of Machine Learning (ML) classifiers where they have been demonstrated to degrade their test time performance. More recently, they have also been demonstrated to degrade the test time performance of trained Reinforcement Learning (RL) policies [3] i.e. an adversary can stealthily modify the observations and cause a trained policy to take incorrect actions. It is essential to address the threats that adversarial examples pose before the widespread adoption of RL policies in critical tasks.

Although there has been a lot of work in crafting adversarial attacks on RL, the number of defenses so far is sparse. In this work, we apply latent representation (feature squeezing by bit depth reduction) of input observations [4] and embed this as a function block in input processing pipeline of a trained policy to defend a trained policy against adversarial attacks. This is an application of existing techniques from image classification domain, where an adversarial example detector was build using this technique, to RL more specifically Deep Q Networks (DQN).

We successfully demonstrate that this technique can defend against an adversarial attack crafted using Fast Gradient Sign Method (FGSM) to a DQN agent trained to play Pong in the Arcade Learning Environment (ALE) [5].

Submitted to Dr. Weizi Li as part of the Reinforcement Learning class project submission.

II. Related Work

In this section, we first introduce existing work on adversarial examples in RL, then we introduce use of feature squeezing as a defense against adversarial examples in image classification.

A. Adversarial examples in Reinforcement Learning

The vulnerability of Reinforcement Learning to adversarial examples was first exposed by Huang et. al. [3], where they discovered that adversarial examples can cause significant drop in performance of policies trained using state of the art algorithms, Deep Q Networks (DQN), Trust Region Policy Optimization (TRPO) and Asynchronous Actor Critic (A3C). They perform their experiments in both white-box and black-box scenarios to further show that adversarial examples crafted for one algorithm can transfer to other algorithm as well. Other attacks techniques such as "Strategically timed attack" [6], where adversarial examples are crafted only for certain time steps independent of all other time steps have then followed. More recently, adversarial examples have also been crafted for automatic path planning tasks [7], [8] .

Although defense techniques like Adversarial training and Defensive distillation have been widely used and validated for image classification, they have not seen widely adopted to adversarial examples in RL [9]. Traditionally, there have been are two techniques to perform a defense against adversarial examples, one of them is to build a detector of adversarial examples whereas other is to use techniques to make it harder to construct an adversarial examples (usually, by modifying the objective function). In this work, we use neither of these techniques i.e. we add a defensive functional block which can cause the trained agent to take correct decision in the presence of an adversary.

B. Feature squeezing in image classification

Feature squeezing was proposed as a detector of adversarial examples by Xu et. al. [4], where they compare the prediction of DNN on a regular input as well as the feature squeezed input. The given input is detected as an adversarial example if the difference in prediction vector as measured by L1 distance between prediction on regular input and

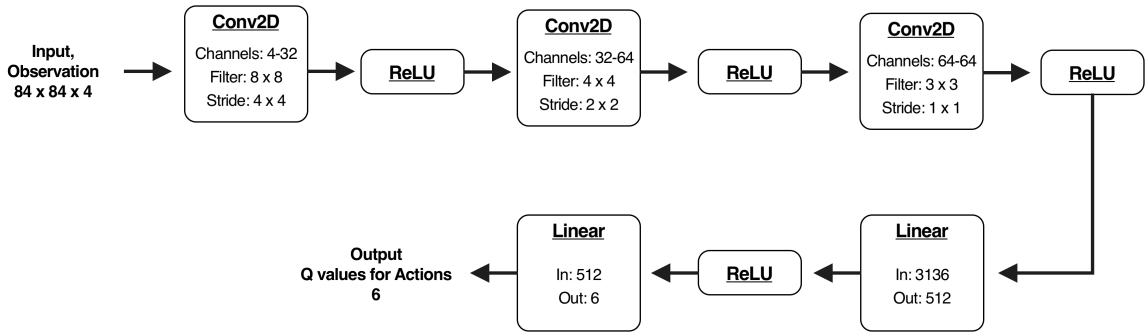

Fig. 1: Architecture of the DQN. The input observation of four stacked $84 \mathbf{x} 84$ pixels is passed though Convolutional layers and Linear layers before mapping them to Q values for the six actions to be performed.

prediction on feature squeezed input is above a predefined threshold. The techniques used by the authors for feature squeezing are bit depth reduction and spatial smoothing and they validated this to be an effective defense technique against adversarial examples crafted using seven different techniques. Although feature squeezing has been used in image classification to detect adversarial examples it has not been use in RL to robustify a trained model.

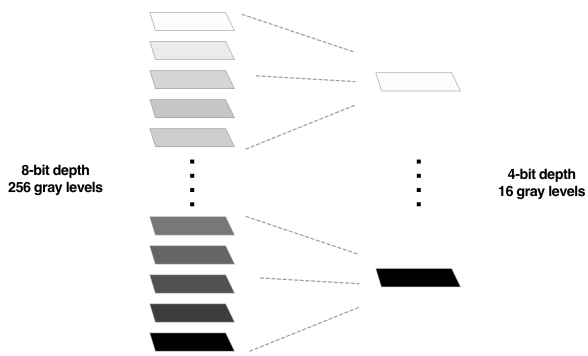

Fig. 2: Feature squeezing by pixel bit depth reduction. The input observation typically represented by 8-bit pixel i.e. 256 levels is reduced to 4-bit pixels i.e. 16 levels.

III. Preliminaries

In this section, we briefly introduce the policy learning algorithm that was attacked, the concept of adversarial attack, the algorithm used to generate adversarial examples and defense technique.

A. Deep Q Networks (DQN)

Introduced in [10] as a variant of the Q learning algorithm, DQN is the generalized approximation of the Q function which achieved state of the art performance in Atari 2600 games from the Arcade Learning Environment. During training, DQN learns to map the raw observations (input pixels) over the distribution of actions to take. Once trained, a policy is obtained which can select optimal action given an observation. One of the games where it achieved superhuman performance was Pong.

The architecture of DQN which is trained to play Pong and craft adversarial examples in this work is shown in Fig. 1 where, the input is a stack of four $84 \mathbf{x} 84$ pixels which is passed through different layers (Convolutional, Activations, Linear) and finally mapped to the Q values of the six actions to take, namely: FIRE, NOOP, RIGHT, RIGHTFIRE, LEFT and LEFTFIRE. Finally, the action taken is the action with the highest Q value and the policy that is learned is obtained by greedily taking actions with highest Q value for each observation.

B. Adversarial Attack

An adversarial attack on deep neural network f is conducted through crafting an adversarial signal $\mathbf{x}^*$ by imposing minimal perturbation $\delta_{\mathbf{x}}$ to the original signal $\mathbf{x}$. $\mathbf{x}^*$ can be obtained via solving the following optimization problem [2]:

$$\begin{aligned} \text{minimize} \quad & D(\mathbf{x}, \mathbf{x}+\delta_{\mathbf{x}}) \\ \text{s.t.} \quad & f(\mathbf{x}) \neq f(\mathbf{x}^*), \end{aligned} \tag{1}$$

where D is a distance metric such as L_0, L_2 or L_∞. Multiple methods can be used to solve Eq. 1, one of them is Fast Gradient Sign Method (FGSM) [11]. FGSM is designed to produce an adversarial signal in a fast, non-iterative manner. It computes the gradient of the cost function J w.r.t input $\mathbf{x}$ and scales the sign of the gradient by a L_∞ constraint for generating $\delta_{\mathbf{x}}$:

$$\mathbf{x}^* = \mathbf{x} + \varepsilon \cdot sign\left(\nabla_x J\left(\theta, \mathbf{x}, y\right)\right), \tag{2}$$

where θ represents the parameters of f; y is the target; and ε is the L_∞ constraint parameter controlling the attack magnitude. Increasing the value of ε will increase the attack effectiveness but also make the adversarial example more distinguishable (i.e., having large $\delta_{\mathbf{x}}$).

C. Feature Squeezing using bit depth reduction

Although neural networks (differentiable models) assume the inputs to them to be continuous, typically, images are quantized to 8-bit representations in the digital form i.e. pixels in each channel have 256 levels leading to 24-bit color in the RGB scheme and 8-bit color in the Grayscale scheme. Xu et. al. [4] first hypothesized that reducing the color depth

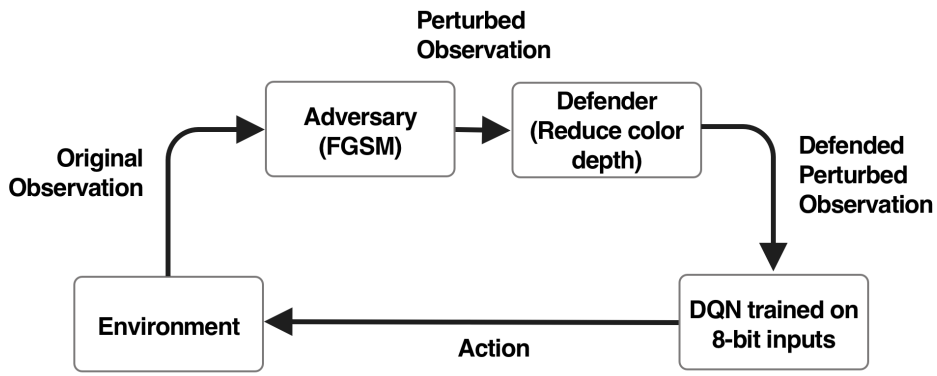

Fig. 3: Systematic diagram of the attack and defense. Adversarial example is generated to the observation obtained from the environment which is then defended before it is fed as an input to trained DQN.

of the input representations can reduce the opportunity for adversary to cause harm without affecting the classifier.

As shown in Fig. 2 feature squeezing coalesces a number of similar inputs into a single one, reducing the high dimensional space of input to its latent representation. Since adversarial examples work by adding fine details to the a given input, in this work, the reduction in pixel depth is expected to eliminate its effects.

IV. Methodology

The defense is implemented against a non adaptive white-box adversary i.e. who knows everything about the model but not the defense technique that is being implemented. The goal of the adversary is to modify the observation to cause a trained model to produce incorrect action whereas the goal of the defense is to eliminate the effects of changes produced by adversary i.e. make the agent take correct action in the presence of adversary.

A. Training DQN

The reward in the game of Pong is defined as $+1$ if a round of game is won and -1 if the round of game is lost. Each episode is composed of 21 rounds i.e. any player can get a maximum reward of $+21$ and the minimum reward of -21. DQN is trained with experience replay following the algorithm given in [10]. As a pre-processing step, to reduce the computational load, the input observation obtained from the ALE which contains RGB frames of size $210 \mathbf{x} 160$ are gray scaled, cropped to relevant area of the frame and resized to $84 \mathbf{x} 84$.

Other training hyper-parameters are: Discount factor $= 0.99$, Learning rate $= 10^{-4}$, Batch size $= 32$, replay buffer size $= 10000$ transactions, Optimizer $=$ Adam [12], Loss $=$ Mean Squared Error (MSE). Further, the agent starts training with a 100% probability of exploration which is gradually decays to a minimum of 2% probability of exploration as it gets better performance. The training is terminated when the mean reward of the last 21 games is $+19$.

B. Generating Adversarial Examples for trained DQN

The white-box adversary has access to the input processing pipeline as shown in Fig. 3, adversarial examples are generated using FGSM to each preprocessed frame obtained from the environment. The perceptibility value is set to 0.003 and the loss function maximized by adversary is MSE. When perturbations are added to the observation, to a human observer the two images are indistinguishable.

C. Using Feature squeezing as a defense

Feature squeezing by color bit depth reduction is applied to image frames perturbed by adversary. The defense mechanism is represented as the 'Latent Representation' block which is embedded in the input processing pipeline of the DQN as shown in Fig. 4. The latent representation essentially works as a filter that prevents the tiny perturbations from being processed at the input.

V. Experiments

In this section, we first describe the experiment set-up and then present and discuss the experiment results. All experiments are conducted using an Intel(R) Core(TM) i7-10700 CPU, a Nvidia RTX 2080 SUPER GPU, and 32G RAM. PyTorch [13] is used to implement the DQN and Advertorch [14] is used to generate adversarial examples.

After the DQN was trained till the convergence criteria defined in Section IV, it was run for a total of 20 episodes. The average reward collected by the agent is given in Table. I. The agent can take correct actions for almost all the rounds of games in every episode.

Number of Episodes	Average Reward
20	+19.75

TABLE I: Performance of a trained DQN on standard observations. As seen from the reward, on every episode, the agens wins almost all rounds of the game.

Following the performance on standard observations, adversarial perturbations were added to the input observation

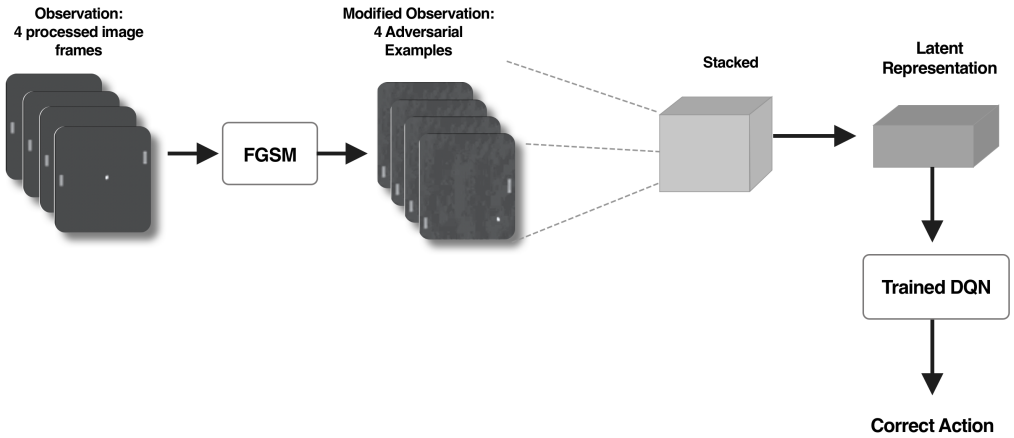

Fig. 4: Defense mechanism embedded to the input processing pipeline. The 'Latent Representation' functional block performs the feature squeezing step on observations that were perturbed by the adversary.

Number of Episodes	Average Reward
20	-21

TABLE II: Performance of a trained DQN on adversarial observations. The agent loses all rounds in every episode of the game that it plays.

Number of Episodes	Average Reward
20	+21

TABLE III: Performance of a trained DQN in presence of 'Latent representation' functional block. The average reward obtained per episode is similar to standard inputs.

frames to generate modified observations as shown in Fig. 4. Table II shows the performance of trained DQN on modified (adversarial) observations. The adversary can significantly degrade the performance of the trained agent i.e. causing it to lose every round of the game in every episode.

The modified observations from the adversary are then passed to a functional block where the input features are squeezed to 4-bit pixel depth reduction before passing it as observations to the DQN which was trained on 8-bit observations. Table III shows the performance of DQN on these defended inputs. Despite the presence of adversary, the agent can perform to win almost all rounds of game in every episode.

VI. Conclusion and Future Work

In this work, we embed a functional block that performs the latent representation of input and acts as a filter to nullify the effects of a white-box adversary to the input processing pipeline of a DQN. Please refer to VII for videos regarding performance. For the future, we plan to use latent representations techniques other than feature squeezing such as Autoencoders to the same problem. In addition, we plan to explore other problems as well.

References

[1] M. Riedmiller, "Neural fitted q iteration–first experiences with a data efficient neural reinforcement learning method," in *European Conference on Machine Learning*. Springer, 2005, pp. 317–328.

[2] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," *arXiv preprint arXiv:1312.6199*, 2013.

[3] S. Huang, N. Papernot, I. Goodfellow, Y. Duan, and P. Abbeel, "Adversarial attacks on neural network policies," *arXiv preprint arXiv:1702.02284*, 2017.

[4] W. Xu, D. Evans, and Y. Qi, "Feature squeezing: Detecting adversarial examples in deep neural networks," *arXiv preprint arXiv:1704.01155*, 2017.

[5] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *Journal of Artificial Intelligence Research*, vol. 47, pp. 253–279, 2013.

[6] Y.-C. Lin, Z.-W. Hong, Y.-H. Liao, M.-L. Shih, M.-Y. Liu, and M. Sun, "Tactics of adversarial attack on deep reinforcement learning agents," *arXiv preprint arXiv:1703.06748*, 2017.

[7] J. Liu, W. Niu, J. Liu, J. Zhao, T. Chen, Y. Yang, Y. Xiang, and L. Han, "A method to effectively detect vulnerabilities on path planning of vin," in *International Conference on Information and Communications Security*. Springer, 2017, pp. 374–384.

[8] X. Bai, W. Niu, J. Liu, X. Gao, Y. Xiang, and J. Liu, "Adversarial examples construction towards white-box q table variation in dqn pathfinding training," in *2018 IEEE Third International Conference on Data Science in Cyberspace (DSC)*. IEEE, 2018, pp. 781–787.

[9] T. Chen, J. Liu, Y. Xiang, W. Niu, E. Tong, and Z. Han, "Adversarial attack and defense in reinforcement learning-from ai security view," *Cybersecurity*, vol. 2, no. 1, pp. 1–22, 2019.

[10] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.

[11] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.

[12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

[13] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," 2017.

[14] G. W. Ding, L. Wang, and X. Jin, "AdverTorch v0.1: An adversarial robustness toolbox based on pytorch," *arXiv preprint arXiv:1902.07623*, 2019.

VII. Appendix A

Click on the items below for videos at various stages of input and the DQN performance:

- Standard and pre-processed inputs
- Performance of trained DQN on standard inputs
- Performance of trained DQN on inputs perturbed by adversary
- Performance of trained DQN on defended inputs